```
    .get()
    .uri(BOOKS_ROOT_API + isbn)
    .retrieve()
    .bodyToMono(Book.class)
    .timeout(Duration.ofSeconds(3), Mono.empty())
    .retryWhen(                                          ❶
      Retry.backoff(3, Duration.ofMillis(100))
    );
}
```

❶ Exponential backoff is used as the retry strategy. Three attempts are allowed with a 100 ms initial backoff.

UNDERSTANDING HOW TO USE RETRIES EFFECTIVELY

Retries increase the chance of getting a response back from a remote service when it's momentarily overloaded or unresponsive. Use them wisely. In the context of timeouts, I highlighted the need for handling read and write operations differently. When it comes to retries, this is even more critical.

Idempotent requests like read operations can be retried without harm. Even some write requests can be idempotent. For example, a request to change the author of a book with a given ISBN from "S.L. Cooper" to "Sheldon Lee Cooper" is idempotent. You could perform it a few times, but the outcome will not change. You shouldn't retry non-idempotent requests, or you'll risk generating inconsistent states. When you order a book, you don't want to be charged multiple times just because the first attempt failed due to the response being lost in the network and never received.

When retries are configured in a flow where the user is involved, remember to balance resilience and user experience. You don't want users to wait too long while retrying the request behind the scenes. If you can't avoid that, make sure you inform the users and give them feedback about the status of the request.